

Autodesk® Scaleform®

Scaleform

C++, , Scaleform 3.1()

.

: Mustafa Thamer

: 2.01

: 2010 9 21

Copyright Notice

Autodesk® Scaleform® 4.4

© 2014 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and other countries: 123D, 3ds Max, Algor, Alias, AliasStudio, ATC, AutoCAD LT, AutoCAD, Autodesk, the Autodesk logo, Autodesk 123D, Autodesk CAM 360, Autodesk Homestyler, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSketch, AutoSnap, AutoTrack, Backburner, Backdraft, Beast, BIM 360, Burn, Buzzsaw, CADmep, CAiCE, CAMduct, CFdesign, Civil 3D, Cleaner, Combustion, Communication Specification, Configurator 360™, Constructware, Content Explorer, Creative Bridge, Dancing Baby (image), DesignCenter, DesignKids, DesignStudio, Discreet, DWF, DWG, DWG (design/logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DWGX, DXF, Ecotect, ESTmep, Evolver, FABmep, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, ForceEffect, FormIt, Freewheel, Fusion 360, Glue, Green Building Studio, Heidi, Homestyler, HumanIK, i-drop, ImageModeler, Incinerator, Inferno, InfraWorks, InfraWorks 360, Instructables, Instructables (stylized robot design/logo), Inventor, Inventor HSM, Inventor LT, Kynapse, Kynogon, LandXplorer, Lustre, MatchMover, Maya, Maya LT, Mechanical Desktop, MIMI, Mockup 360, Moldflow Plastics Advisers, Moldflow Plastics Insight, Moldflow, Moondust, MotionBuilder, Movimento, MPA (design/logo), MPA, MPI (design/logo), MPX (design/logo), MPX, Mudbox, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, Pixlr, Pixlr-omatic, Productstream, Publisher 360, RasterDWG, RealDWG, ReCap, ReCap 360, Remote, Revit LT, Revit, RiverCAD, Robot, Scaleform, Showcase, Showcase 360 ShowMotion, Sim 360, SketchBook, Smoke, Socialcam, Softimage, Sparks, SteeringWheels, Stitcher, Stone, StormNET, TinkerBox, ToolClip, Topobase, Toxik, TrustedDWG, T-Splines, ViewCube, Visual LISP, Visual, VRED, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. "AS IS." AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

:

Scaleform Autodesk Scaleform Corporation 6305 Ivy Lane, Suite 310 Greenbelt, MD 20770, USA www.scaleform.com info@scaleform.com (301) 446-3200 (301) 446-3199

1	C++, ,	1
1.1	C++	2
1.1.1	FSCCommand	3
1.1.2	API.	4
1.2	C++	8
1.2.1		8
1.2.2		10
1.2.3	(Paths)	11
2	Direct Access API	14
2.1		15
2.2		17
2.3		17
2.4	Direct Access	21
2.4.1	Object	21
2.4.2	Array	22
2.4.3		23
2.4.4	사항	23
2.4.5		24

1 C++, ,

. ,

,

, ,

.

. Scaleform

C++

. , , ,

C++

.

C++

.

ActionScript	C++	C++	ActionScript
FSCommand		Gfx::Movie::Get/SetVariable	
ExternalInterface		Gfx::Movie::Invoke	
Direct Access Objects			
Gfx::Value			

1.1

C++

Scaleform C++ *FSCommand*

ExternalInterface 2 . API

. FSCommand ExternalInterface C++

. Scaleform

GFX::Loader, GFX::MovieDef , GFX::Movie .

Scaleform (states) .

FSCommand 'fscommand' . Fscommand

2 , .

C++ 2 . C++ fscommand

fscommand .

ExternalInterface *flash.external.ExternalInterface.call*

. C++

ExternalInterface GFX::Value . C++

ExternalInterface GFX::Value ExternalInterface.call .

ExternalInterface API , . (

Scaleform)

FSCommands ExternalInterface

. .

FSCommands ExternalInterface Direct Access API GFX::FunctionHandler

. ActionScript VM C++

. ActionScript Invoke C++

Invoke . [Direct Access API](#) GFX::FunctionHandler

. .

1.1.1 FSCommand

```
fscommand

bool

fscommand("setMode", "2");

Gfx::Loader, Gfx::MovieDef Gfx::Movie
    Gfx::Loader -> Gfx::MovieDef -> Gfx::Movie
    (override)
Gfx::Movie Gfx::RenderConfig( EdgeAA ) Gfx::Translator(
    )
    (delegation chain) ( Gfx::Loader ).
Gfx::Movie FSCommand
GfxPlayerTiny ("FxpPlayerFSCommandHandler" ).

fscommand Gfx::FSCommandHandler

class OurFSCommandHandler : public FSCommandHandler
{
public:
    virtual void Callback(Movie* pmovie, const char* pcommand,
        const char* parg)
    {
        Printf("FSCommand: %s, Args: %s", pcommand, parg);
    }
};
```

```

                                fscommand                2
fscommand                                .                fscommand

                                .

Gfx::Loader                                .

// Register our fscommand handler
Ptr<FSCommandHandler> pcommandHandler = *new OurFSCommandHandler;
gfxLoader->SetFSCommandHandler(pcommandHandler);

Gfx::Loader                                Gfx::Movie    Gfx::MovieDef
                                SetFSCommandHandler

C++                                (non-blocking)

                                Advance    Invoke

```

1.1.2 API

```

ExternalInterface.call                fscommand

. ExternalInterface                fscommand

ExternalInterface

class OurExternalInterfaceHandler : public ExternalInterface
{
public:
    virtual void Callback(Movie* pmovieView, const char* methodName,
                          const Value* args, unsigned argCount)
    {
        Printf("ExternalInterface: %s, %d args: ", methodName, argCount);
        for(unsigned i = 0; i < argCount; i++)
        {
            switch(args[i].GetType())
            {
            case Value::VT_Number:
                Printf("%3.3f", args[i].GetNumber());
                break;
            case Value::VT_String:

```



```

        Printf("%s", args[i].GetString());
        break;
    }
    Printf("%s", (i == argCount - 1) ? "" : ", ");
}
Printf("\n");

// return a value of 100
Value retValue;
retValue.SetNumber(100);
pmovieView->SetExternalInterfaceRetVal(retValue);
}
};

```

GfX::Loader

```

Ptr<GFxExternalInterface> pEIHandler = *new OurExternalInterfaceHandler;
gfxLoader.SetExternalInterface(pEIHandler);

```

ExternalInterface

1.1.2.1 FxDelegate

ExternalInterface

ExternalInterface

```

, ( ) GfX::Value
FxDelegate (Apps\Samples\GameDelegate\FxGameDelegate.h )

```

FxDelegate GfXExternalInterface . Register/Unregister

FxDelegate . FxDelegate

[HUD Kit](#)

FxDelegate

```

// Minimap Begin
void FxPlayerApp::Accept(FxDelegateHandler::CallbackProcessor* cbreg)
{

```

```

cbreg->Process("registerMiniMapView", FxPlayerApp::RegisterMiniMapView);
cbreg->Process("enableSimulation", FxPlayerApp::EnableSimulation);
cbreg->Process("changeMode", FxPlayerApp::ChangeMode);
cbreg->Process("captureStats", FxPlayerApp::CaptureStats);

cbreg->Process("loadSettings", FxPlayerApp::LoadSettings);
cbreg->Process("numFriendliesChange", FxPlayerApp::NumFriendliesChange);
cbreg->Process("numEnemiesChange", FxPlayerApp::NumEnemiesChange);
cbreg->Process("numFlagsChange", FxPlayerApp::NumFlagsChange);
cbreg->Process("numObjectivesChange", FxPlayerApp::NumObjectivesChange);
cbreg->Process("numWaypointsChange", FxPlayerApp::NumWaypointsChange);
cbreg->Process("playerMovementChange", FxPlayerApp::PlayerMovementChange);
cbreg->Process("botMovementChange", FxPlayerApp::BotMovementChange);

cbreg->Process("showingUI", FxPlayerApp::ShowingUI);
}

```

FxDelegate

1.1.2.2

```

import flash.external.*;
ExternalInterface.call("foo", arg);

```

```

'foo'          'arg'          .          0
. ExternalInterface API

```

ExternalInterface

ExternalInterface

```

flash.external.ExternalInterface.call("foo", arg)

```

1.1.2.3 ExternalInterface.addCallback

ExternalInterface.addCallback

C++ `ExternalInterface::Movie::Invoke()` (this)

```

import flash.external.*;

var txtField:TextField = this.createTextField("txtField",
                                             this.getNextHighestDepth(), 0, 0, 200, 50);
var aliasName:String = "setText";
var instance:Object = txtField;
var method:Function = SetText;
var wasSuccessful:Boolean = ExternalInterface.addCallback(aliasName, instance,
                                                         method);

function SetText()
{
    trace(this + ".SetText ");
    this.text = "INVOKED!";
}

```

this SetText txtField

C++ :
 pMovie->Invoke("setText", "");

"txtField.SetText" , txtField "INVOKED!"

Gfx::Movie::Invoke() 1.2.2

1.2 C++

C++
 . , Scaleform C++
 . Scaleform C++ get/set
 (, ,).
 Direct Access API , C++
 ActionScript . [Direct Access API](#)

1.2.1

Scaleform [GetVariable](#) [SetVariable](#) ,
 . 2 Direct Access API
 . Set/GetVariable
 Direct Access API .
 Direct Access
 GFX::Value::SetMember . ,
 . , GetVariable .
 GFX::Value , Direct Access .
 GetVariable SetVariable .

```
int counter = (int)pHUDMovie->GetVariableDouble("_root.counter");
counter++;
pHUDMovie->SetVariable("_root.counter", Value((double)counter));
```

 GetVariableDouble _root.counter . C++
 . GetVariableDouble 0 .

```
counter                                _root.counter    SetVariable                                .
```

GetVariable SetVariable [GFX::Movie](#)

```
SetVariable    GFx::Movie::SetVarType    ,    "sticky"
```

```

3         _root.mytextfield
4
5     SetVariable("_root.mytextfield.text", "testing", SV_Normal)

```

```
1
SV_Sticky ( )
    _root.mytextfield.text
3
C++
SV_Normal
```

SV_Sticky SV_Normal .

Scaleform [SetVariableArray](#) [GetVariableArray](#)

SetVariableArray

Gfx::Movie::SetVariableArraySize

SetVariableArray

```
int idxInASArray = 0;
const char* strarr[2];
strarr[0] = "This is the first string";
strarr[1] = "This is the second one";
pMovie->SetVariableArray(Movie::SA_String, "_root.strArray",
                        idxInASArray, strarr, 2);
```

wide

```
int idxInASArray = 0;
const wchar_t* strarr[2];
strarr[0] = L"This is the first string";
```

```
strarr[1] = L"This is the second one";
pMovie->SetVariableArray(Movie::SA_StringW, "_root.strArray",
                        idxInASArray, strarr, 2)
```

GetVariableArray

AS

1.2.2

[GFX::Movie::Invoke\(\)](#)

, UI

UI

Direct Access API

:

mySlider' _ root . SetSliderPos mySlider'

:

```
this.SetSliderPos = function (pos)
{
    // Clamp the incoming position value
    if (pos<rangeMin) pos = rangeMin;
    if (pos>rangeMax) pos = rangeMax;
    gripClip._x = trackClip._width * ((pos-rangeMin)/(rangeMax-rangeMin));
    gripClip.gripPos = gripClip._x;
}
```

gripClip mySlider pos rangeMin/Max

C++ :

```
Value result;
bool bInvoked = pMovie->Invoke("_root.mySlider.SetSliderPos", &result, "%d",
                                newPos);
```

```
Invoke                                true                                false                                .
                                     . Invoke
```

Scaleform	.	.	1
GFX::Movie::Advance	initFirstFrame	true	
GFX::MovieDef::CreateInstance	.		

Invoke `Gfx::Movie::IsAvailable()` Invoke

```
Value result;
if (pMovie->IsAvailable("parentPath.mySlider.SetSliderPos"))
    pMovie->Invoke("parentPath.mySlider.SetSliderPos", &result, "%d", newPos);
```

```
printf'      Invoke      Invoke
      .
      GfX::Value
```

```
Value args[3], result;
args[0].SetNumber(i);
args[1].SetString("test");
args[2].SetNumber(3.5);
pMovie->Invoke("path.to.methodName", &result, args, 3);
```

```
InvokeArgs                                     va_list
      . Invoke  InvokeArgs  printf  vprintf
```

1.2.3 (Paths)

GfX::Movie

```

Movie::IsAvailable(path)
Movie::SetVariable(path, value)
Movie::SetVariableDouble(path, value)
Movie::SetVariableArray(path, index, data, count)
Movie::SetVariableArraySize(path, count)
Movie::GetVariable(value, path)
Movie::GetVariableDouble( path)
Movie::GetVariableArray(path, index, data, count)
Movie::GetVariableArraySize(path)
Movie::Invoke(pathToMethodName, result, argList, ...)

```

(.)

"clip2"

"clip1"

→ clip1 → clip2

1. clip1

2. clip2 clip1

```

"clip1.clip2"
"_root.clip1.clip2"
"_level0.clip1.clip2"

```

"Clip1.clip2" ← . Clip1

"clip2" ← clip1

ActionScript2 "_root" "_level0"

. ActionScript 3

ActionScript 3

. _root, _level0, root, level0.

SWF

_root

_levelN .

- (Ctrl+Enter), (**Ctrl+Alt+V**)()

- 1 .
.), (**Ctrl+Alt+V**)()

- loadMovie
_level0 _level1 _level2
().
_levelN.objectName _levelN.objectPath.objectName (N)

2 .

1. [Paths to Objects and Variables](#) Jesse Stratford
2. [Advanced Pathing](#) Jesse Stratford

2 Direct Access API

GFx::Value Direct Access Scaleform 3.1
API ()
()
UI Direct Access API
HUD [Minimap Demo](#)

Direct Access API GFx::Values
(, ,) GFx::Value API
GFx::Movie API
[GFx::Value](#)

Direct Access API (GFx::Value)C++

GFx::Movie

Direct Access API GFx::Movie
[GFx::Value](#)
root 'foo'
Invoke Invoke

1. GFx::Movie Invoke ():

```
Value ret;
Value args[N];
pMovie->Invoke("_root.foo.method", &ret, args, N);
```

2. Direct Access Invoke ():

```
(foo _root.foo )
Value ret;
Value args[N];
bool = foo.Invoke("method", &ret, args, N);
```

```
Invoke      Direct Access API      get, set      ,
      Gfx::Value::HasMember, GetMember, SetMember      .
      (Gfx::Value      )movieClip      GetMember
      .
```

```
Value tf;
MovieClip.GetMember("textField", &tf);
tf.SetText("hello");
```

```
movieClip      textField      ,      SetText
      .
```

2.1

Direct Access API

C++

2

```
Movie* pMovie = ... ;
Value parent, child;
pMovie->CreateObject(&parent);
pMovie->CreateObject(&child);
parent.SetMember("child", child);
```

CreateObject

:

```
Value mat, params[6];
// (set params[0..6] to matrix data) ...
pMovie->CreateObject(&mat, "flash.geom.Matrix", params, 6);

// (Assuming pMovie is a Gfx::Movie*):
Value owner, childArr, childObj;
pMovie->CreateArray(&owner);           // create parent array
pMovie->CreateArray(&childArr);         // create child array
pMovie->CreateObject(&childObj);        // create child object
bool = owner.SetElement(0, childArr);  //set parent[0] to child array
bool = owner.SetElement(1, childObj);  // set parent[1]to child object
...
bool = foo.SetMember("owner", owner); // later, set the 'owner' variable in
// object foo, to the parent object
```

Gfx::Value::VisitMembers()

```
public
ObjectVisitor
    (override)
    . VisitMembers
    ( ) (enumerable)
    ASSetPropFlags
```

e.g.: `_global.ASSetPropFlags(MyClass.prototype, ["someFunc", "__get__number", "test"], 6, 1);`

```
(getter/setter) ASSetPropFlags VisitMembers
    (enumerable) , VT_Object
(ASSetPropFlags ) VT_Object
```

2.2

```

Direct Access API . DisplayObject
    ,
    .
    DisplayInfo API . DisplayInfo
    , , , , ,
    .
SetMember . SetDisplayInfo
    .
    Gfx::Movie::SetVariable
    .
    SetDisplayInfo
    .

// Assumes MovieClip is a Gfx::Value*
Value::DisplayInfo info;
GPointF pt(100,100);
info.SetRotation(90);
info.SetPosition(pt.x, pt.y);
MovieClip.SetDisplayInfo(info);

```

2.3

```

ActionScript2 .
(Introspection: )
    . Gfx::Movie::CreateFunction , C++
    . CreateFunction VM
    . AS , C++
    (fscommand ExternalInterface )
    VM
    .
    AS
    .

Value obj;
pmovie->GetVariable(&obj, "_root.obj");

```

```

class MyFunc : public FunctionHandler
{
    public:
        virtual void Call(const Params& params)
        {
            // Callback logic/handling
        }
};
Ptr<MyFunc> customFunc = *SF_HEAP_NEW(Memory::GetGlobalHeap()) MyFunc();
Value func;
pmovie->CreateFunction(&func, customFunc);
obj.SetMember("func", func);

```

```

        ActionScript                                . (_root                                )

obj.func(param1, param2, param3);

```

```

Params                                Call()                                .

• pRetVal:                                Gfx::Value                                .

        ,                                pMovie->CreateObject()

        CreateArray()                                .                                (Numbers, Booleans    )                                setter

        .

• pMovie:                                .

• pThis:                                .                                undefined

        .

• ArgCount:

• pArgs:                                Gfx::Value                                . []

        .

        : Value firstArg = params.pArgs[0];

• pArgsWithThisRef:

        .                                . (

        .)

• pUserData:                                .

        (delegation)                                .

```



```
pmovie->SetVariable("_global.Network", networkCtorFn);
```

VM

```
var netObj:Object = new Network(param1, param2);
```

VM

VM

```
Value origFuncReal;
obj.GetMember("funcReal", &origFuncReal);
class FuncRealIntercept : public FunctionHandler
{
    Value OrigFunc;
public:
    FuncRealIntercept(Value origFunc) : OrigFunc(origFunc) {}
    virtual ~FuncRealIntercept() {}
    virtual void Call(const Params& params)
    {
        // Intercept logic (beginning)
        OrigFunc.Invoke("call", params.pRetVal, params.pArgsWithThisRef,
                        params.ArgCount + 1);
        // Intercept logic (end)
    }
};
Value funcRealIntercept;
Ptr<FuncRealIntercept> funcRealDef = *SF_HEAP_NEW(Memory::GetGlobalHeap())
                                     FuncRealIntercept(origFuncReal);
pmovie->CreateFunction(&funcRealIntercept, funcRealDef);
obj.SetMember("funcReal", funcRealIntercept);
```

: VM

.swf(GFx::Movie)

VM

GFX::Value

2.4 Direct Access

Direct Access API public [. Gfx::Value](#)

2.4.1 Object

	Public
	('foo' "_root.foo" Object)
	bool has = foo.HasMember("bar");
	Value bar; bool = foo.GetMember("bar", &bar);
	Value val; bool = foo.SetMember("bar", val);
	Value ret; Value args[N]; bool = foo.Invoke("method", args, N, &ret); or foo.Invoke("method", args, N);
	Value obj; pMovie->CreateObject(&obj); ... bool = foo.SetMember("bar", obj);
	Value owner, child; pMovie->CreateObject(&owner); pMovie->CreateObject(&child); bool = owner.SetMember("child", child); ... bool = foo.SetMember("owner", owner);
	Value::ObjectVisitor v; foo.VisitMembers(&v);
	bool = foo.DeleteMember("bar");

2.4.2 Array

Public	
('bar' " _root.foo.bar" Array 'foo' Object)	
	<code>unsigned sz = bar.GetArraySize();</code>
	<code>Value val; bool = bar.GetElement(idx, &val);</code>
	<code>Value val; bool = bar.SetElement(idx, val);</code>
	<code>bool = bar.SetArraySize(N);</code>
	<code>Value arr; pMovie->CreateArray(&arr); ... bool = foo.SetMember("bar", arr);</code>
	<code>Value owner, childArr, childObj; pMovie->CreateArray(&owner); pMovie->CreateArray(&childArr); pMovie->CreateObject(&childObj); bool = owner.SetElement(0, childArr); bool = owner.SetElement(1, childObj); ... bool = foo.SetMember("owner", owner);</code>
	Enumeration <code>for (UPInt i=0; i <N; i++) { ... bool = bar.GetElement(i+idx, &val) ... }</code> Using a visitor pattern: <code>Value::ArrayVisitor v; bar.VisitElements(v, idx, N);</code>
	<code>bool = bar.ClearElements();</code>
	PushBack <code>Value val; bool = bar.PushBack(val);</code> PopBack <code>Value val; bool = bar.PopBack(&val); or void bar.PopBack();</code>
	Single element <code>bool = bar.RemoveElement(idx);</code> Series of elements <code>bool = bar.RemoveElements(idx, N);</code>

2.4.3

	(‘foo’ (MovieClip, TextField, Button) . “_root.foo.bar”)
.	Value::DisplayInfo info; bool = foo. GetDisplayInfo (&info);
.	Value::DisplayInfo info; ... bool = foo. SetDisplayInfo (info);
.	Matrix2_3 mat; bool = foo. GetDisplayMatrix (&mat);
.	Matrix2_3 mat; ... bool = foo. SetDisplayMatrix (mat);
.	Render::Cxform cxform; bool = foo. GetColorTransform (&cxform);
.	Render::Cxform cxform; ... bool = foo. SetColorTransform (cxform);

2.4.4

사항

	(‘foo’ . “_root.foo”)
.	Value newInstance; bool = foo. AttachMovie (&newInstance, "SymbolName", "instanceName");
.	Value emptyInstance; bool = foo. CreateEmptyMovieClip (&emptyInstance, "instanceName");
.	bool = foo. GotoAndPlay ("myframe");
.	bool = foo. GotoAndPlay (3);
.	bool = foo. GotoAndStop ("myframe");
.	bool = foo. GotoAndStop (3);

2.4.5

		('foo'	["_root.foo"])
	·	Value text; bool = foo.GetText(&text);	
	·	Value text; ... bool = bar.SetText(text);	
HTML	·	Value htmlText; bool = bar.GetTextHTML (&htmlText);	
HTML	·	Value htmlText; ... bool = bar.SetTextHTML(htmlText);	